

Übungsklausur 2 — Web-Engineering 1

Stil: Dozenten-Probeklausur (TINF21B2) · 75 Punkte · Bearbeitungszeit ~60 Min · MUSTERLÖSUNG

Aufgabe 1 (HTML)

25 Punkte

a) 4 P: Beschreiben Sie den Aufbau (die vollständige Struktur) eines HTML-Tags. Gehen Sie auch auf leere Auszeichnungen ein.

Lösung: Normal: `<name attribut="wert">Inhalt</name>` (Start-Tag + Attribute + Inhalt + End-Tag). Leere (void) Elemente haben keinen Inhalt und kein End-Tag, z.B. `<br>`, `<img>`, `<hr>`.

b) 4 P: Nennen Sie zwei Tags, die in den HTML-Head gehören, und erklären Sie, was diese Tags bewirken.

Lösung: `<title>` = Titel im Browser-Tab/Suchergebnis. `<meta charset="UTF-8">` = Zeichenkodierung. Auch `<link rel="stylesheet">` (CSS), `<script>` (JS).

c) 2 P: Erstellen Sie ein Hyperlink-Element, das auf „https://dhw.de“ zeigt und den Text „DHBW“ anzeigt.

Lösung: `<a href="https://dhw.de">DHBW</a>`

d) 4 P: Erstellen Sie ein HTML-Fragment (ohne CSS): eine Überschrift „Woche“ und darunter ein Absatz mit einem kursiven Wort.

Lösung: `<h3>Woche</h3><p>Heute ist <em>Montag</em>.</p>`

e) 3 P: Erstellen Sie ein HTML-Fragment für eine geordnete Liste (1.2.3.) mit drei Einträgen.

Lösung: `<ol><li>Erstens</li><li>Zweitens</li><li>Drittens</li></ol>`

f) 4 P: Was ist eine Definitionsliste? Geben Sie ein Beispiel-Fragment mit einem Begriff an.

Lösung: Dritte Listenart, ordnet Begriffen Erklärungen zu: `<dl><dt>HTTP</dt><dd>Übertragungsprotokoll des Webs</dd></dl>`

g) 4 P: Benennen Sie einen HTTP-Nummernkreis-Statuscode und erklären Sie, wann er auftritt. Nennen Sie außerdem einen Bestandteil einer URL.

Lösung: z.B. **404** (4xx Client-Fehler): angeforderte Ressource nicht gefunden. URL-Bestandteil: Protokoll/Host/Pfad, z.B. Host `dhw.de`.

Aufgabe 2 (CSS)

18 Punkte

a) 4 P: Geben Sie ein Beispiel für eine valide CSS-Regel mit existierenden Eigenschaften und erfundenen Werten an.

Lösung: z.B. `p { color: #123abc; font-size: 42px; }`

b) 2 P: Beschreiben Sie eine Möglichkeit, CSS in ein HTML-Dokument einzubinden.

Lösung: Intern über ein `<style>`-Tag im `<head>` — zentrale Stelle für Style-Angaben einer Seite.

c) 4 P: Geben Sie für das folgende HTML-Dokument den „top“- und „left“-Wert aller drei div-Elemente in Pixel an.

```
#a { position:absolute; top:20px; left:30px; width:80px; height:80px; }
#b { position:relative; top:5px; left:5px; width:80px; height:80px; }
#c { top:99px; left:99px; width:80px; height:80px; }

<div id="a"><div id="b"></div></div>
<div id="c"></div>
```

Lösung: **#a** (absolute): top **20px**, left **30px**. **#b** (relative, in #a): Normalpos (30,20) + top5/left5 → top **25px**, left **35px**. **#c** (kein position → static): top/left wirkungslos, und #a ist aus dem Fluss → top **0px**, left **0px**.

d) 4 P: Geben Sie eine syntaktisch-korrekte CSS-Regel mit mehreren Selektoren und einer Eigenschaft an (erfundene Werte).

Lösung: `h1, h2, .titel { color: teal; }`

e) 4 P: Gegeben sei der folgende HTML-Body. Welche Elemente (IDs) sprechen die Selektoren an?

```
<h2 id="h">H</h2>
<p id="p1" class="a">1</p>
<ol id="o" class="liste wichtig">
  <li id="x1">a</li>
  <li id="x2"><span id="sp">b</span></li>
</ol>
<p id="p2" class="wichtig">2</p>
<span id="sp2">z</span>
```

Selektoren: `.wichtig` · `ol[class="liste wichtig"]` · `ol[class="wichtig"]`
 · `[class~="wichtig"]` · `#x2` · `:only-child` · `li:first-child` · `ol > li` · `li + li`

Lösung: `.wichtig` → p2, o, sp · `ol[class="liste wichtig"]` → o (exakt) · `ol[class="wichtig"]` → (nichts) ·
`[class~="wichtig"]` → p2, o, sp · `#x2` :only-child → sp · `li:first-child` → x1 · `ol > li` → x1, x2 · `li + li` → x2

Aufgabe 3 (JavaScript)

15 Punkte

a) 2 P: Nennen Sie ein Beispiel für einen Zugriff, den der Schutzmechanismus Sandbox verhindert.

Lösung: z.B. eine Webseite kann per JS NICHT eine lokale Datei vom Rechner lesen, oder die Daten/Cookies einer fremden Domain auslesen (Same-Origin-Policy).

b) 2 P: Beschreiben Sie einen Weg, JavaScript in eine HTML-Webseite einzubinden, und geben Sie ein Code-Fragment an.

Lösung: Extern via `<script src="app.js"></script>` oder intern via `<script> /* code */ </script>`.

c) 2 P: Benennen und beschreiben Sie ein Objekt, das im Browser als Standard-API zur Verfügung steht.

Lösung: z.B. `document` — Zugriff auf das DOM (Elemente finden/ändern). Auch `console`, `window`, `fetch`.

d) 5 P: Gegeben sei folgendes Codesegment. Welchen Wert haben x1–x5? Objekt-/Array-Werte in JSON-Notation.

```
let a="Web"; let b=100;
let c = {"a":1,"b":2,"c":3,"d":4};
function Summe(d,e){
  let a=0;
  b = d.length*10;
  for(const k in e){ a = a+e[k]; }
  this.v=a; this.w=d+"!";
  return a;
}
let x1 = new Summe(a,c);
let x2 = Summe("Test",c);
let x3 = a; let x4 = [c]; let x5 = b;
```

Lösung: `x1 = {"v":10,"w":"Web!"}` (new → Objekt) · `x2 = 10` · `x3 = "Web"` · `x4 = [{"a":1,"b":2,"c":3,"d":4}]` · `x5 = 40` (globales b zuletzt: "Test".length*10 = 40).

e) 4 P: Geben Sie ein JavaScript-Fragment an, das einen Listeneintrag mit dem Inhalt „Montag“ und dem id-Attribut „tag“ erzeugt und in eine bestehende Liste mit der id „woche“ einfügt.

Lösung: `const li=document.createElement("li"); li.id="tag"; li.textContent="Montag";`
`document.getElementById("woche").appendChild(li);`

Aufgabe 4 (WebServer)

4 Punkte

a) 2 P: Benennen Sie zwei Aufgaben, die ein WebServer übernimmt.

Lösung: Zwei von: HTTP-Requests entgegennehmen und beantworten; statische Ressourcen ausliefern bzw. an Programmcode delegieren (dynamische Erzeugung); Zugriffe protokollieren.

b) 2 P: In welchem Prozessmodell wird CGI verwendet? Beschreiben Sie die Funktionsweise.

Lösung: CGI läuft im Modell **getrennter Prozesse**: Pro Anfrage startet der Webserver einen eigenen, neuen Prozess, der die Ausgabe (HTML) erzeugt und zurückliefert — isoliert, aber mit Prozess-Start-Overhead.

Aufgabe 5 (NodeJS)

13 Punkte

a) 2 P: Beschreiben Sie den Unterschied zwischen einem serverinternen Forward und einem Redirect des Clients/Browsers.

Lösung: Forward: der Server reicht die Anfrage intern weiter, die URL im Browser bleibt gleich, 1 Roundtrip. Redirect: der Server antwortet mit 3xx + neuer URL, der Browser stellt eine zweite Anfrage, die URL ändert sich.

b) 4 P: Schreiben Sie eine Middleware: Bei Anfrage auf „/anfrage“ prüfen, ob der Body den Parameter „param“ mit Wert „42“ hat. Wenn ja, Weiterleitung zu „/spezial“, sonst normal weiterlaufen lassen.

Lösung: `app.use("/anfrage", (req, res, next) => { if (req.body.param == "42") res.redirect("/spezial"); else next(); });`

c) 3 P: Beschreiben Sie die Funktionsweise von Pragmas/Templates und was als Ergebnis entsteht, nachdem das Template gerendert wurde.

Lösung: Ein Template enthält HTML mit Platzhaltern (Pragmas). Beim Rendern werden die Platzhalter durch konkrete Daten ersetzt; Ergebnis ist fertiges, statisches HTML, das an den Client gesendet wird.

d) 4 P: Wie erweitern Cookies das HTTP-Request-Response-Paar? Nennen Sie außerdem die zwei Gültigkeitsbereiche von Variablen in NodeJS.

Lösung: Cookies: Server sendet `Set-Cookie`, der Browser speichert es und hängt es bei jedem passenden Folge-Request automatisch an → macht das zustandslose HTTP zustandsbehaftet. NodeJS-Scopes: Modul-lokal und global.